

[Open in app](#) ↗Access to everything. Now 30% off. [Upgrade now](#)

Spsc Queue Implementation +

Low-Latency C++ Applications Series — Part 6: `std::atomic`, Memory Models, and SPSC Queue Implementation

15 min read · Sep 6, 2026



Kemal Candan

[Follow](#)

Listen



Share



More



When designing multi-threaded and low-latency systems, moving away from locks (mutexes) toward lock-free architectures makes `std::atomic` the most fundamental building block provided by C++. However, one of the most common misconceptions I encounter is believing that simply adding `std::atomic` to a variable will magically solve all synchronization issues without sacrificing performance.

In reality, sticking to default settings when using `std::atomic` can sometimes cause performance losses at the microarchitectural level. In this article, we will move beyond the theoretical side of C++ memory models and examine what actually happens at the hardware level (Store Buffers, x86 TSO, and ARM Weak Ordering), how the compiler reorders our code, and finally, take a deep dive into my low-latency SPSC Queue implementation.

The Big Misconception: Atomicity ≠ Ordering

When you define a variable as `std::atomic<T>`, the C++ standard gives you only one guarantee: read or write operations on that variable are indivisible. In other words, when another thread reads this variable, it will not see half of the data in an old state and half in a new state (a torn read). Atomic accesses work through a specific atomic object's modification order without causing data races.

However, the critical point is this: **atomicity says nothing about the order of this operation relative to other memory accesses around it.**

This is exactly where “Memory Order” comes into play. The order of the code we write is not the same as the order the processor executes it. Two different forces constantly try to reorder our code:

- **Compiler Level (Compiler Reordering):** Under the “as-if” rule, your compiler (GCC, Clang) can swap, combine, or cache instructions in registers as long as the final output of the program within a single thread remains unchanged.
- **Hardware Level (CPU Out-of-Order Execution):** Modern processors execute instructions out of order to keep the execution pipeline full. Furthermore, CPU cores have their own “Store Buffers” and “Load Queues”. While you think you are writing directly to memory, the data might actually be waiting in a Store Buffer and may not yet be visible to other cores.

This illusion works perfectly in a single thread. However, in HFT systems, a sequence where you say “I wrote A, then I wrote B” might appear as “B arrived first, then A” to another CPU core communicating over L1 caches. Memory order constraints determine how strictly we limit this hardware and compiler chaos.

C++ Memory Order Categories and the Happens-Before Relationship

The C++ standard provides various `memory_order` arguments to control these reorderings. To understand them, grasping the “happens-before” concept is

essential. In the C++ memory model, happens-before establishes a strong ordering relationship between two evaluations. Thanks to this relationship, previous memory effects are guaranteed to be observable by the subsequent thread.

The primary ordering constraints we use in low-latency systems are as follows:

std::memory_order_seq_cst (Sequential Consistency)

If you do not specify any `memory_order`, the C++ compiler uses `seq_cst` by default. In addition to acquire/release semantics, it provides a single total order across all `seq_cst` atomic operations. This stronger guarantee can require a more expensive implementation on certain architectures. Therefore, it is important to avoid using an ordering stronger than what is actually needed in the hot path.

std::memory_order_relaxed

This is the weakest constraint, but the most valuable for performance. It only guarantees atomicity. While preserving atomicity and modification-order guarantees for the atomic operation itself, it does not provide inter-thread ordering for other memory accesses. It is mostly used when there is no causal connection to other memory states, such as counters or metric increments (e.g., `packets_received.fetch_add(1, std::memory_order_relaxed);`). In HFT systems, the majority of "pure state" updates are typically done using `relaxed`.

std::memory_order_acquire and std::memory_order_release

This is the heart of lock-free data structures.

- **memory_order_release**: When a thread writes to an atomic variable with `release`, no read or write operations prior to this line can be moved after it.
- **memory_order_acquire**: When a thread reads the same atomic variable with `acquire`, no read or write operations following this line can be moved before it.

This pair forms a perfect “Producer / Consumer” mechanism. For example, when you prepare data (`data = compute()`) and then set a flag (`ready.store(true, release)`), another thread that observes the flag (`ready.load(acquire)`) knows with hardware guarantees that the data is fully written to memory and ready for use.

How Memory Order Translates to Assembly via Compiler Explorer: x86-64 vs ARM64

The `std::memory_order` rules we abstract at the C++ level meet the processor's architectural memory model (Memory Consistency Model) during compilation. Today, HFT and AI infrastructures run not only on traditional x86-64 (Intel/AMD) architectures but also on ARM64 (aarch64) server architectures such as AWS Graviton or Ampere.

Let's take a closer look at the assembly instructions generated by the compiler across these different architectures.

Simple C++ Test Code Snippet

Let's define the load, store, and Read-Modify-Write (RMW) operations in our application at the C++ level as follows:

```
#include <atomic>
#include <cstdint>

std::atomic<uint64_t> a{0};

// LOAD
uint64_t load_relaxed() { return a.load(std::memory_order_relaxed); }
uint64_t load_acquire() { return a.load(std::memory_order_acquire); }
uint64_t load_seqcst() { return a.load(std::memory_order_seq_cst); }

// STORE
void store_relaxed(uint64_t v) { a.store(v, std::memory_order_relaxed); }
void store_release(uint64_t v) { a.store(v, std::memory_order_release); }
void store_seqcst(uint64_t v) { a.store(v, std::memory_order_seq_cst); }

// RMW
uint64_t add_relaxed(uint64_t v) { return a.fetch_add(v, std::memory_order_relaxed); }
uint64_t add_seqcst(uint64_t v) { return a.fetch_add(v, std::memory_order_seq_cst); }
```

Compiler-Generated Assembly Outputs

x86-64 Assembly (GCC 16.1 -O3)

```
"load_relaxed()":
    mov     rax, QWORD PTR "a"[rip]
    ret
"load_acquire()":
    mov     rax, QWORD PTR "a"[rip]
    ret
```

```

"load_seqcst()":
    mov     rax, QWORD PTR "a"[rip]
    ret
"store_relaxed(unsigned long)":
    mov     QWORD PTR "a"[rip], rdi
    ret
"store_release(unsigned long)":
    mov     QWORD PTR "a"[rip], rdi
    ret
"store_seqcst(unsigned long)":
    xchg    rdi, QWORD PTR "a"[rip]
    ret
"add_relaxed(unsigned long)":
    mov     rax, rdi
    lock xadd     QWORD PTR "a"[rip], rax
    ret
"add_seqcst(unsigned long)":
    mov     rax, rdi
    lock xadd     QWORD PTR "a"[rip], rax
    ret

```

ARM64 Assembly (GCC 16.1 -O3)

```

load_relaxed():
    adrp    x0, .LANCHOR0
    add     x0, x0, :lo12:.LANCHOR0
    ldr     x0, [x0]
    ret
load_acquire():
    adrp    x0, .LANCHOR0
    add     x0, x0, :lo12:.LANCHOR0
    ldar    x0, [x0]
    ret
load_seqcst():
    adrp    x0, .LANCHOR0
    add     x0, x0, :lo12:.LANCHOR0
    ldar    x0, [x0]
    ret
store_relaxed(unsigned long):
    adrp    x1, .LANCHOR0
    add     x1, x1, :lo12:.LANCHOR0
    str     x0, [x1]
    ret
store_release(unsigned long):
    adrp    x1, .LANCHOR0
    add     x1, x1, :lo12:.LANCHOR0
    stlr    x0, [x1]
    ret

```

```

store_seqcst(unsigned long):
    adrp    x1, .LANCHOR0
    add     x1, x1, :lo12:.LANCHOR0
    stlr    x0, [x1]
    ret

add_relaxed(unsigned long):
    stp     x29, x30, [sp, -16]!
    adrp    x1, .LANCHOR0
    add     x1, x1, :lo12:.LANCHOR0
    mov     x29, sp
    bl      __aarch64_ldadd8_relax
    ldp     x29, x30, [sp], 16
    ret

add_seqcst(unsigned long):
    stp     x29, x30, [sp, -16]!
    adrp    x1, .LANCHOR0
    add     x1, x1, :lo12:.LANCHOR0
    mov     x29, sp
    bl      __aarch64_ldadd8_acq_rel
    ldp     x29, x30, [sp], 16
    ret
    .set    .LANCHOR0, . + 0

```

Hardware-Level Engineering Takeaways

Examining the assembly outputs from a microarchitectural perspective reveals three critical facts that low-latency software developers should know:

A. x86-64 TSO and the “Free Acquire/Release” Illusion

The x86 hardware inherently has a strict memory model (Total Store Order). The processor already executes Store-Store and Load-Load operations sequentially at the hardware level; the only allowed out-of-order execution is Store-Load reordering.

- This causes `relaxed`, `acquire`, and `seq_cst` reads on x86 to compile into the exact same instruction (`mov`).
- On the store side, a `release` store is also just a simple `mov` operation. Only a `seq_cst` store forces the hardware to place a full barrier, causing the compiler to generate an `xchg` instruction. `xchg` is an instruction that requires stronger synchronization and atomicity semantics compared to a plain store, and its observed cost depends on the microarchitecture, cache state, and surrounding code.

B. The ARM64 Weak Model Reality: Barriers Are Not Free

The ARM64 architecture allows wide reordering freedom at the hardware level to maximize performance.

- When performing an acquire read on ARM, `ldar` is generated instead of a plain `ldr`, and when performing a release write, `stlr` is produced.
- `LDAR` and `STLR` are special atomic memory-access instructions carrying load-acquire and store-release ordering semantics, respectively. Therefore, memory orderings chosen casually on x86 assuming "it doesn't matter anyway" can cause measurable tail latency increases when migrating to ARM-based servers (such as Apple Silicon or AWS Graviton).

C. The RMW (`fetch_add`) Misconception: Compiler Freedom vs. Hardware Instruction

The common perception among developers that “using `relaxed fetch_add` on x86 will run faster” is partially incorrect.

- **On x86:** Whether `fetch_add` is `relaxed` or `seq_cst`, the generated assembly instruction does not change (`lock xadd`). This is because the `lock` prefix already forms a full fence in hardware. The performance gain here does not come from the instruction itself, but from the compiler's freedom to reorder surrounding independent C++ code when `relaxed` is chosen.
- **On ARM64:** In ARM's LSE (Large System Extensions) architecture, genuinely different instructions are invoked at the hardware level (such as `ldadd` vs. `ldaddal`). The chosen memory order on ARM64 can change the atomic instruction or instruction sequence selected by the compiler, resulting in different execution costs.

Now that we clearly understand the weight of atomic variables in the microarchitecture and the dangers of reordering, we can combine these building blocks (Acquire/Release semantics) to move on to that famous structure that allows us to communicate between two CPU cores: the SPSC (Single-Producer Single-Consumer) Lock-Free Queue.

SPSC Queue Implementation

The implementation is split into separate modules for the queue, platform-specific utilities, and benchmark harness. To keep the article focused on the data structure

itself, the snippet below shows the core `SPSCQueue` implementation; the complete implementation is available in the accompanying repository.

Github link: <https://github.com/kemalcandann/spsc-queue>

```
template <typename T, std::size_t Capacity>
class SPSCQueue {
private:
    static constexpr std::size_t Mask = Capacity - 1;

    struct Node {
        alignas(alignof(T))
        std::byte storage[sizeof(T)];
    };

    Node buffer_[Capacity];

    alignas(std::hardware_destructive_interference_size)
    std::atomic<std::size_t> tail_{0};

    std::size_t head_cached_{0};

    alignas(std::hardware_destructive_interference_size)
    std::atomic<std::size_t> head_{0};

    std::size_t tail_cached_{0};

}
```

```
template <typename... Args>
bool try_emplace(Args&&... args)
    noexcept(std::is_nothrow_constructible_v<T, Args&&...>)
{
    const std::size_t tail = tail_.load(std::memory_order_relaxed);

    if (tail - head_cached_ == Capacity) {

        head_cached_ = head_.load(std::memory_order_acquire);

        if (tail - head_cached_ == Capacity) {
            return false;
        }
    }

    std::construct_at(
        object_ptr(buffer_[tail & Mask]),
        std::forward<Args>(args)...
```

```

    );

    tail_.store(tail + 1, std::memory_order_release);

    return true;
}

```

```

bool try_pop(T& value)
    noexcept(std::is_nothrow_move_assignable_v<T> && std::is_nothrow_destructible_v<T>)
{
    static_assert(
        std::is_nothrow_move_assignable_v<T>,
        "T must be nothrow move assignable");

    static_assert(
        std::is_nothrow_destructible_v<T>,
        "T must be nothrow destructible");

    const std::size_t head = head_.load(std::memory_order_relaxed);

    if (head == tail_cached_) {

        tail_cached_ = tail_.load(std::memory_order_acquire);

        if (head == tail_cached_) {
            return false;
        }
    }

    T* item = object_ptr(buffer_[head & Mask]);

    value = std::move(*item);

    std::destroy_at(item);

    head_.store(head + 1, std::memory_order_release);

    return true;
}

```

Understanding the SPSC Design

The implementation above is intentionally specialized for the **Single-Producer / Single-Consumer (SPSC)** case.

The key property of an SPSC queue is that there is exactly one producer and exactly one consumer. This allows us to assign ownership of the two queue indices:

```
Producer owns → tail_  
Consumer owns → head_
```

The producer is the only thread that advances `tail_`, while the consumer is the only thread that advances `head_`. Because each index has a single writer, the queue does not need CAS-based coordination such as `compare_exchange` to update these indices.

This is one of the main reasons SPSC queues can be significantly simpler than general-purpose MPMC queues.

Ring-buffer indexing

The queue uses monotonically increasing `head_` and `tail_` counters instead of storing only the current array position.

Because the capacity is a power of two, the physical slot can be calculated efficiently with:

```
index & (capacity - 1)
```

Using monotonic counters also allows queue occupancy to be represented directly as:

```
tail - head
```

Unsigned integer wrap-around is naturally handled by the counter arithmetic.

How Memory Order Is Used in the SPSC Queue

Above, we discussed memory ordering in terms of the general C++ memory model. Here, we apply these same principles directly to the queue.



Because the producer owns its `tail_` value, it can read it with `relaxed`. The consumer, on the other hand, observes the `tail_` value with `acquire` before reading the object created by the producer:

```

Producer

construct T
|
v
tail_.store(release)
|
| synchronization
v
Consumer

tail_.load(acquire)
|
v
read T

```

In the reverse direction, after the consumer consumes the slot and destroys the object, it publishes the `head_` value with `release`. When the producer observes this value with `acquire`, the slot becomes available for reuse:

```

Consumer

read / destroy T
|
v
head_.store(release)
|
| synchronization
v
Producer

```

```
head_.load(acquire)
|
v
reuse slot
```

The key idea here is to use `relaxed` for owner-local state, and `acquire` / `release` for publishing and reclaiming data between threads.

Cached remote indices

Another important design choice is that each thread keeps a local copy of the index owned by the other thread:

```
Producer:
    tail_          -> local state
    head_cached_   -> cached consumer state
```

```
Consumer:
    head_          -> local state
    tail_cached_   -> cached producer state
```

The producer does not need to read `head_` on every `push()`. It first checks its cached value and refreshes it only when the queue may be full.

Likewise, the consumer only refreshes `tail_` when its cached value indicates that the queue may be empty.

This keeps the normal fast path mostly local and reduces unnecessary cross-core cache-coherency traffic.

Why use raw storage?

The ring buffer stores raw memory:

```
std::byte storage[sizeof(T)];
```

rather than constructing a `T` object in every slot at initialization. This separates **memory storage** from **object lifetime**. A slot becomes a live `T` object with:

```
std::construct_at(...)
```

After the consumer finishes using the element, its lifetime ends with:

```
std::destroy_at(...)
```

This lets the queue control object lifetime explicitly without requiring every slot to contain a default-constructed object.

False sharing

The producer-owned and consumer-owned atomic indices are separated using:

```
alignas(std::hardware_destructive_interference_size)
```

The purpose is to reduce **false sharing**, where independent frequently-modified variables happen to occupy the same cache line.

The producer and consumer still need to communicate through `head_` and `tail_`; this does not remove cache-coherency traffic. The goal is to avoid adding unnecessary contention between unrelated pieces of state.

Benchmark methodology

The benchmark separates two different measurements.

Latency / RTT uses two SPSC queues in a ping-pong configuration:

```
Producer  
|  
| q_tx
```

```
    v
Consumer
  |
  | q_rx
    v
Producer
```

This measures the end-to-end round-trip communication time through two SPSC queues.

Throughput uses a streaming configuration:

```
Producer ---> SPSC Queue ---> Consumer
```

Here the producer continuously publishes messages while the consumer continuously removes them. The goal is to measure the sustained message-processing rate rather than a single message's round-trip time.

The benchmark also uses warm-up phases, CPU affinity, repeated runs, ordered TSC reads, `TSC_AUX` consistency checking, and percentile-based statistics.

Why look beyond the average?

For low-latency systems, the average alone is usually not enough.

A latency distribution is more informative:

```
P50
P90
P99
P99.9
P99.99
Max
```

The median gives us a view of typical behavior, while the higher percentiles reveal the tail of the distribution.

A system can have a very small median latency while still experiencing much larger latency spikes occasionally. For a low-latency system, reducing the tail latency can be as important as reducing the median. For this reason, tail latency is an important part of evaluating low-latency concurrent systems. A low median latency does not necessarily imply deterministic latency.

Benchmark Results

The benchmark was executed with the producer and consumer pinned to separate CPUs.

The system reported the following TSC clocksource calibration:

```
tsc: Refined TSC clocksource calibration: 3792.896 MHz
clocksource: tsc
```

For the results below, this corresponds to approximately **3.792896 GHz**, or about **0.26365 ns per TSC tick**.

Note: TSC ticks are not necessarily CPU core cycles. The TSC is used here as a low-overhead timestamp source, while the calibrated TSC frequency is used to convert the measured deltas into time units.

Latency / RTT

The latency benchmark uses two SPSC queues in a ping-pong configuration. The reported value is therefore a **round-trip time (RTT)** through the two queues and the participating threads, not a one-way queue operation latency.

```
=====
                SPSC LOW-LATENCY BENCHMARK
=====
Producer CPU      : 2
Consumer CPU      : 4
Repetitions       : 5
Latency Queue Capacity: 2
Throughput Queue Size : 65536
TSC Frequency     : 3.792764 GHz
=====
```

[1] LATENCY / RTT

Warm-up samples : 1000000
Measured samples : 2000000

Run 1

Mean : 636 TSC ticks (167.688 ns) TSC ticks
P50 : 646 TSC ticks (170.324 ns)
P90 : 722 TSC ticks (190.362 ns)
P99 : 760 TSC ticks (200.382 ns)
P99.9 : 1482 TSC ticks (390.744 ns)
P99.99 : 8550 TSC ticks (2.254 us)
Max : 89072 TSC ticks (23.485 us)

Run 2

Mean : 649 TSC ticks (171.115 ns) TSC ticks
P50 : 646 TSC ticks (170.324 ns)
P90 : 722 TSC ticks (190.362 ns)
P99 : 1064 TSC ticks (280.534 ns)
P99.9 : 2698 TSC ticks (711.355 ns)
P99.99 : 10374 TSC ticks (2.735 us)
Max : 137446 TSC ticks (36.239 us)

Run 3

Mean : 665 TSC ticks (175.334 ns) TSC ticks
P50 : 646 TSC ticks (170.324 ns)
P90 : 722 TSC ticks (190.362 ns)
P99 : 988 TSC ticks (260.496 ns)
P99.9 : 5928 TSC ticks (1.563 us)
P99.99 : 11172 TSC ticks (2.946 us)
Max : 156446 TSC ticks (41.249 us)

Run 4

Mean : 653 TSC ticks (172.170 ns) TSC ticks
P50 : 646 TSC ticks (170.324 ns)
P90 : 722 TSC ticks (190.362 ns)
P99 : 836 TSC ticks (220.420 ns)
P99.9 : 5206 TSC ticks (1.373 us)
P99.99 : 10754 TSC ticks (2.835 us)
Max : 176472 TSC ticks (46.529 us)

Run 5

Mean : 649 TSC ticks (171.115 ns) TSC ticks
P50 : 646 TSC ticks (170.324 ns)
P90 : 722 TSC ticks (190.362 ns)
P99 : 1102 TSC ticks (290.553 ns)
P99.9 : 2850 TSC ticks (751.431 ns)
P99.99 : 9348 TSC ticks (2.465 us)
Max : 175142 TSC ticks (46.178 us)

Latency summary (median across runs)

Mean : 649 TSC ticks (171.115 ns) TSC ticks
P50 : 646 TSC ticks (170.324 ns)
P90 : 722 TSC ticks (190.362 ns)
P99 : 988 TSC ticks (260.496 ns)

```

P99.9   : 2850 TSC ticks (751.431 ns)
P99.99  : 10374 TSC ticks (2.735 us)
Max     : 156446 TSC ticks (41.249 us)

```

The benchmark results establish a strong un-tuned baseline, delivering a median P50 round-trip time (RTT) of 170.3 ns with deterministic consistency across all five runs. The tight spread between P50 (170.3 ns) and P99 (260.5 ns) reflects an exceptionally predictable latency profile under continuous load. The tail latency spikes — reaching 2.74 μ s at P99.99 and a maximum of 41.25 μ s — are expected characteristics of unmitigated OS jitter, such as hardware interrupts, timer ticks, and CPU power-state transitions.

Throughput

```

[2] THROUGHPUT
Warm-up operations   : 50000000
Measured operations  : 50000000

Run 1: 153.60 Mmsg/s
Run 2: 153.54 Mmsg/s
Run 3: 155.35 Mmsg/s
Run 4: 152.45 Mmsg/s
Run 5: 156.57 Mmsg/s

Throughput summary (median across runs): 153.60 Mmsg/s

=====
IMPORTANT:
Latency values are TSC ticks, not CPU core cycles.
TSC_AUX remained constant during each measured run.
=====

```

These numbers should be treated as a **baseline for this implementation and benchmark configuration**, not as a universal claim about SPSC queues or production HFT systems.

The exact result depends on hardware topology, CPU model, compiler version and flags, operating-system configuration, CPU affinity, cache state, interrupt activity, and the benchmark methodology.

Benchmark Limitations

The benchmark is intended to evaluate the implementation and establish a reproducible low-latency baseline rather than represent a production HFT environment.

Latency measurements are reported as TSC-based measurements, and the benchmark uses CPU affinity, warm-up iterations, repeated runs, and percentile statistics. The results should therefore be interpreted together with the exact hardware, compiler, operating-system, CPU topology, and benchmark configuration. All benchmark tests were conducted on an **AMD Ryzen 7 7840HS** processor featuring **8 physical cores (16 threads)**. The memory hierarchy includes **256 KiB L1d**, **256 KiB L1i**, **8 MiB L2**, and a shared **16 MiB L3** cache.

The goal of this experiment is not to claim that this implementation represents the absolute minimum latency achievable by an SPSC queue. Instead, it provides a baseline that can be investigated further using hardware performance counters and operating-system-level tuning.

What I Learned

The most important lesson from this implementation was that lock-free programming is not simply about replacing a mutex with `std::atomic`.

The interesting part is understanding which thread owns which state, which operations actually need synchronization, and how those synchronization points establish the required happens-before relationships.

The SPSC constraint makes this problem much simpler because each queue index has a single writer. That allows the implementation to avoid CAS-based coordination and keep the normal fast path mostly local.

The benchmark also reinforced an important lesson from low-latency engineering: average latency is not enough. The tail of the latency distribution can look very different from the median, which makes percentiles an essential part of the measurement.

Conclusion and What's Next

It is important to emphasize that the benchmark results presented in this post represent a **baseline implementation**. At this stage, we have deliberately not applied any low-latency system-level optimizations or hardware tuning techniques.

This initial run was executed without several standard HFT infrastructure practices, including:

- **Kernel & CPU Tuning:** Core isolation (`isolcpus` , `nohz_full`), strict CPU affinity, or disabling power-saving P-states.
- **Interrupt Routing:** Steering hardware interrupts away from dedicated worker cores using `IRQ_BANNED_CPUS` .
- **Memory Management:** Allocating memory via HugePages to eliminate TLB misses during execution.
- **Compiler & Build Configurations:** Microarchitecture-specific tuning flags or Link-Time Optimization (LTO).

Consider this an entry-level, baseline SPSC queue implementation designed to establish a clean and functionally sound lock-free foundation.

In the next part of the series, we will take this baseline and push it to its limits. We will use profiling tools like `perf` to locate exact CPU bottlenecks and cache misses, apply these OS and kernel-level optimizations step-by-step, and measure how much tail latency we can carve out.

See you in the next article!

References

1. [Single Producer Single Consumer Lock-free FIFO From the Ground Up — Charles Frasch — CppCon 2023](#)
2. [User API & C++ Implementation of a Multi Producer, Multi Consumer, Lock Free, Atomic Queue — CppCon](#)
3. <https://github.com/rigtorp/SPSCQueue>
4. https://www.boost.org/doc/libs/1_78_0/doc/html/boost/lockfree/spsc_queue.htm

SpSC Queue Implementation

